

Design and Development of a Multi-Agent AI System for Training Course Design and Student Evaluation

An Agentic AI System for Intelligent Course Planning, Assessment, Student Performance Analysis, and Personalized Learning Recommendations

(تصميم نظام متعدد الوكلاء بالذكاء الاصطناعي لتصميم الدورات التدريبية وتقييم أداء الطلاب)

Project Designer: Mohammad Abu Alqasem

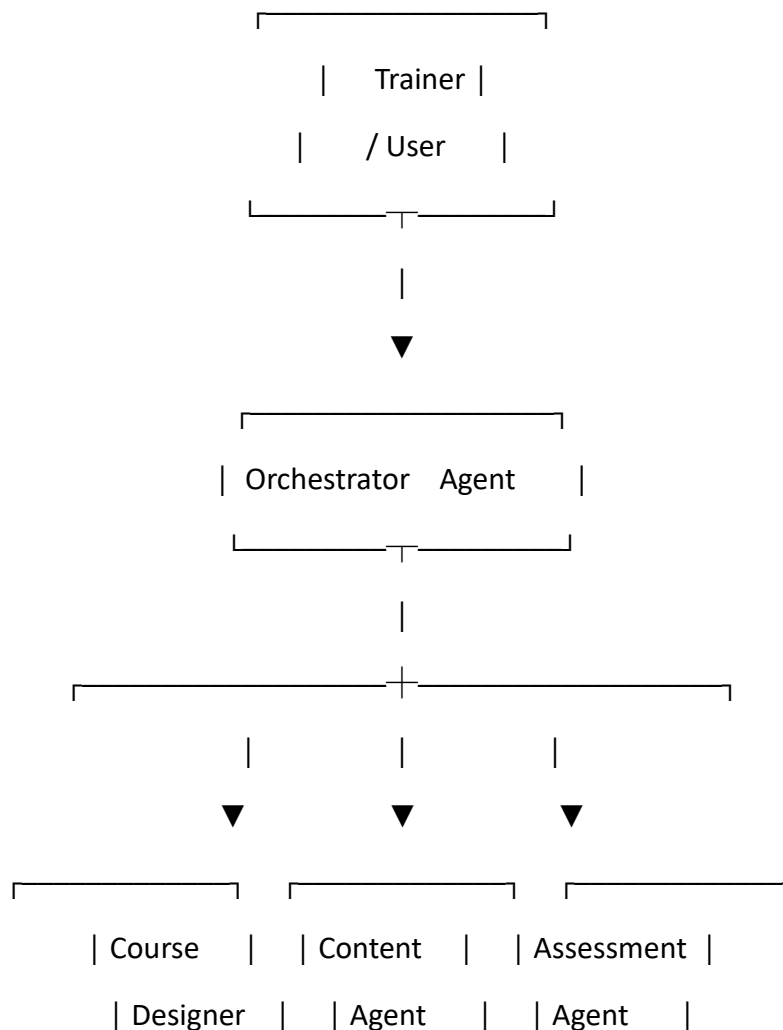
Project Supervisor: Dr. Fakh El-Din Abbas

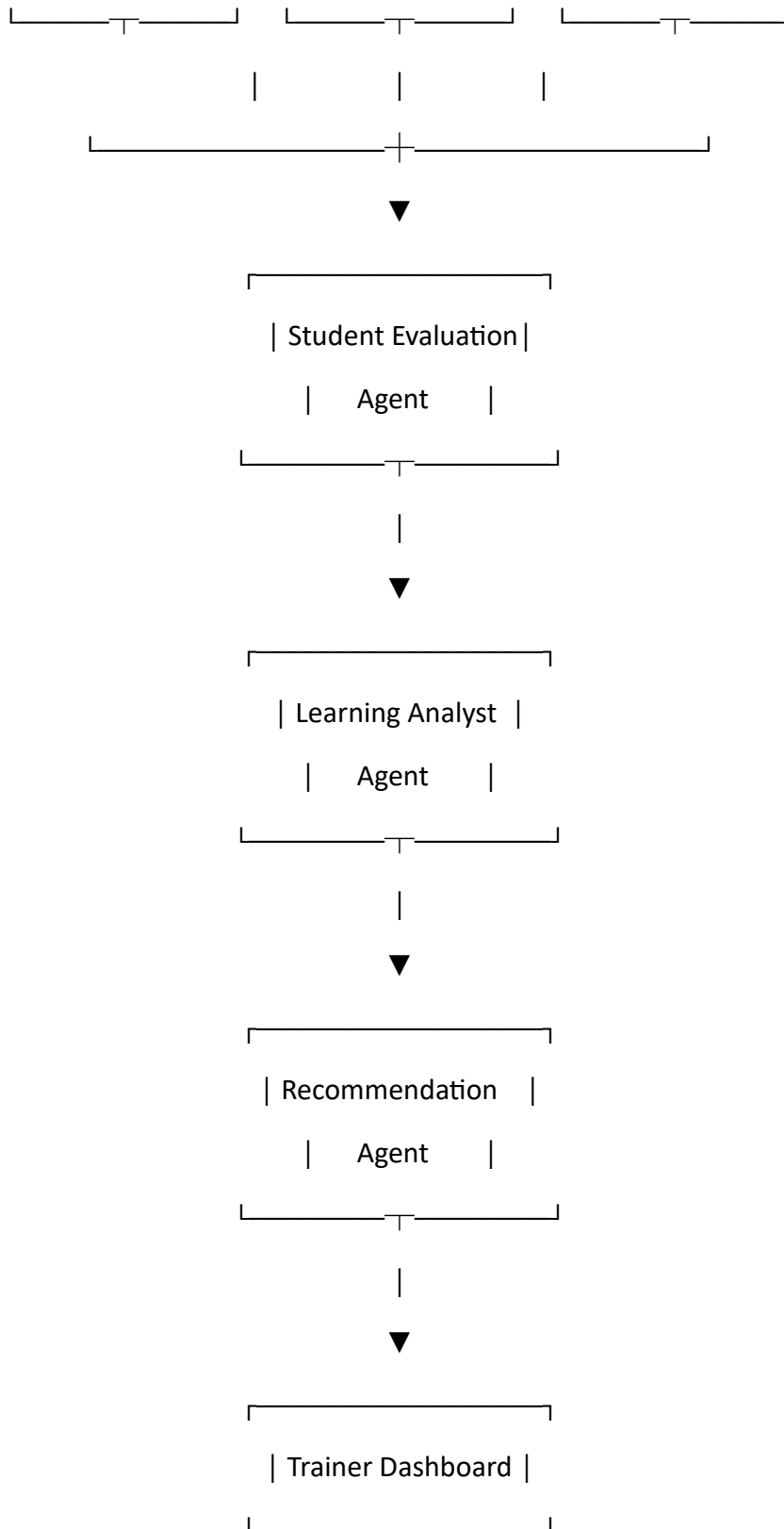
1- فكرة النظام

النظام عبارة عن منصة ذكية تساعد المدرب على:

تصميم الدورة التدريبية ← إنشاء المحتوى والأنشطة ← تقييم الطلاب ← تحليل النتائج ← اكتشاف نقاط الضعف ← تعديل الخطء التدريبية.

الفكرة الأساسية هي أن لدينا عدة Agents ، وكل Agent مسؤول عن مهمة محددة.





- إنشاء المحتوى.
- إعداد الاختبارات.
- تصحيح الاختبارات.
- تحليل نتائج الطلاب.
- اكتشاف الطلاب المتعثرين.

- 2- المشكلة التي يحلها النظام :
- المدرّب عادة يقوم بهذه المهام يدويًا:
- تصميم Course Outline.
 - تقسيم الساعات.
 - كتابة Learning Objectives.

Module 5: Functions	5 hours
Module 6: Data Structures	5 hours
Module 7: Final Project	3 hours

4. Agent 2 : Content Generator Agent

هذا الـ Agent يستلم **Course Plan** من **Course Designer**

ويقوم بإنشاء:

Lecture Topics – Examples- Exercises- Practical Tasks- Learning Materials

مثلاً:

Module: Loops

Learning Objectives:

1. Understand for loops
2. Understand while loops
3. Use loops with lists
4. Apply nested loops

ثم يولد:

Example 1

Example 2

Exercise 1

Exercise 2

Practical Task

5. Agent 3 : Assessment Agent

هذا الـ Agent مسؤول عن الاختبارات. يمكنه إنشاء: **MCQ**

What is the output of:

for i in range(3):

print(i)

- A. 1 2 3
- B. 0 1 2
- C. 0 1 2 3
- D. Error

أنواع الأسئلة

- تحديد المواضيع التي تحتاج إلى إعادة شرح.
- إعداد تقارير الأداء.

النظام سيحول هذه العمليات إلى **Agentic Workflow**.

3- الـ Agents الأساسية :

لدينا 7 Agents :

Agent 1 : Course Designer Agent

مسؤول عن تصميم الدورة.

Input

Course: Python Programming

Level: Beginner

Duration: 30 hours

Students: 20

Goal: Teach students Python programming fundamentals

Output

Course Title

Course Description

Learning Objectives

Modules

Topics

Hours per Module

Prerequisites

Assessment Strategy

مثلاً:

Module	Topic
Module 1: Python Basics	4 hours
Module 2: Variables & Types	4 hours
Module 3: Conditions	4 hours
Module 4: Loops	5hours

Conditions → Strong

Loops → Weak

Functions → Weak

Lists → Average

7. Agent 5 : Learning Analyst Agent

هذا من أهم Agents في النظام.

بدلاً من قول:

الطالب حصل على 60%.

يقول:

الطالب يعاني من ضعف واضح في التعامل مع loops و functions، بينما يظهر أداءً جيداً في variables و conditions.

ثم يحلل المجموعة كلها.

مثلاً:

Class Performance

Variables 87%

Conditions 82%

Loops 54%

Functions 48%

Lists 73%

ويستنتج:

Critical Topic:

Functions

Recommendation:

Trainer should review functions

before moving to advanced topics.

8. Agent 6 : Recommendation Agent

هذا الـ Agent يحول التحليل إلى قرارات تعليمية.

مثلاً:

IF class mastery < 60%

MCQ

True / False

Short Answer

Coding Question

Practical Task

والأهم أن الأسئلة ترتبط بـ Learning Objectives.

مثلاً:

Question Q01

↓

Learning Objective LO-03

↓

Module 2

↓

Python Variables

وهذه نقطة مهمة جداً في المشروع لأنها تسمح للنظام لاحقاً بتحليل أي Learning Objective لم يتحقق بشكل جيد.

6. Agent 4 : Student Evaluation Agent :

يستقبل إجابات الطلاب.

مثلاً:

Student: S001

Q1 = Correct

Q2 = Correct

Q3 = Wrong

Q4 = Wrong

Q5 = Correct

ويحسب:

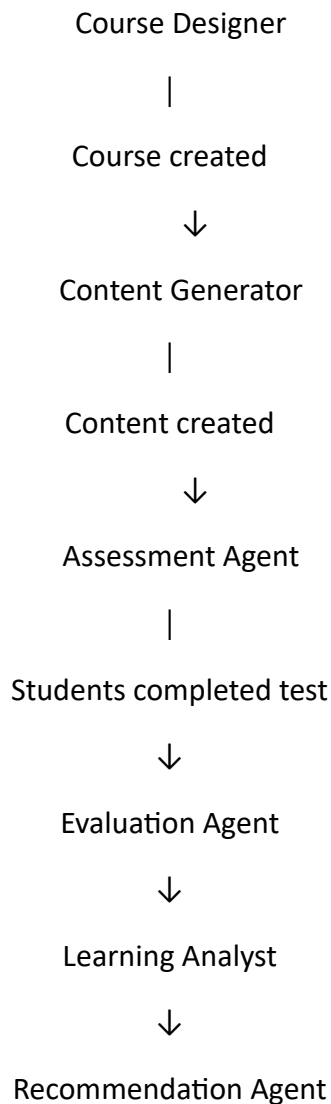
Score = 60%

لكن لا نريد مجرد Score.

نريد : Student Performance Profile

مثلاً:

Variables → Strong



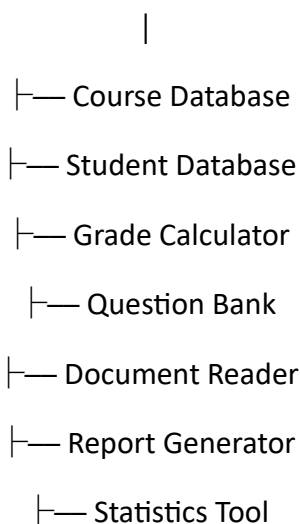
وهذا هو الجزء الذي يجعل النظام **Agentic**.

10. -Tools :

حتى يكون المشروع Agentic فعلاً، يجب ألا تعتمد الـ Agents على LLM فقط.

نحتاج Tools حقيقية. مثلاً:

Tools



→ Recommend reteaching

IF student mastery < 50%

→ Recommend remedial exercises

IF student mastery > 90%

→ Recommend advanced exercises

مثلاً:

Student S001

Weak Areas:

- Loops
- Functions

Recommended Actions:

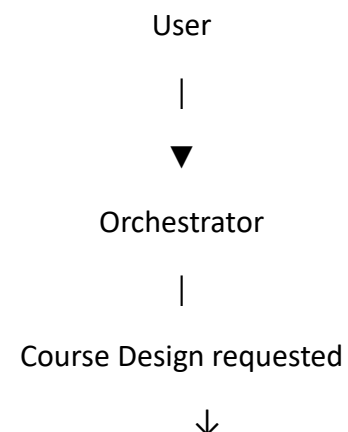
1. Review loops.
2. Complete 5 loop exercises.
3. Watch functions lesson.
4. Complete function practical task.
5. Retake assessment.

9. أهم جزء: Orchestrator Agent

لا نريد أن يعمل كل Agent بشكل مستقل.

نحتاج Agent رئيسياً يسمى: **Orchestrator**

وظيفته تحديد: من يجب أن يعمل الآن؟ مثلاً:



12. Pydantic Schemas

وهنا يمكننا تطبيق ما تعلمناه في كورس

(COSC726 Agentic AI.)

بدلاً من أن يكون الناتج:

The course should have around 7 modules...

نريد. Structured Output.

مثلاً:

```
class LearningObjective(BaseModel):
```

```
    id: str
```

```
    description: str
```

```
    level: str
```

```
class Module(BaseModel):
```

```
    id: str
```

```
    title: str
```

```
    duration_hours: float
```

```
    objectives: list[LearningObjective]
```

```
class CoursePlan(BaseModel):
```

```
    title: str
```

```
    description: str
```

```
    duration_hours: float
```

```
    modules: list[Module]
```

وبالتالي يصبح الناتج قابلاً للمعالجة برمجياً.

13. State :

من النقاط المهمة جداً: النظام يجب أن يحتفظ بحالة العملية. مثلاً

```
class TrainingState(BaseModel):
```

```
    course: CoursePlan | None = None
```

مثلاً: **Grade Calculator Tool**

```
    calculate_score(
```

```
        correct_answers=18,
```

```
        total_questions=25
```

```
)
```

يعطي:

72%

الـ LLM لا ينبغي أن يقوم بالحساب بنفسه إذا كان بإمكان Tool موثوق تنفيذ ذلك.

11. Database

يمكن أن نبدأ بـ SQLite

الجداول الأساسية:

courses

students

modules

learning_objectives

questions

assessments

student_answers

student_results

recommendations

العلاقات تقريباً:

Course

|

└─ Modules

|

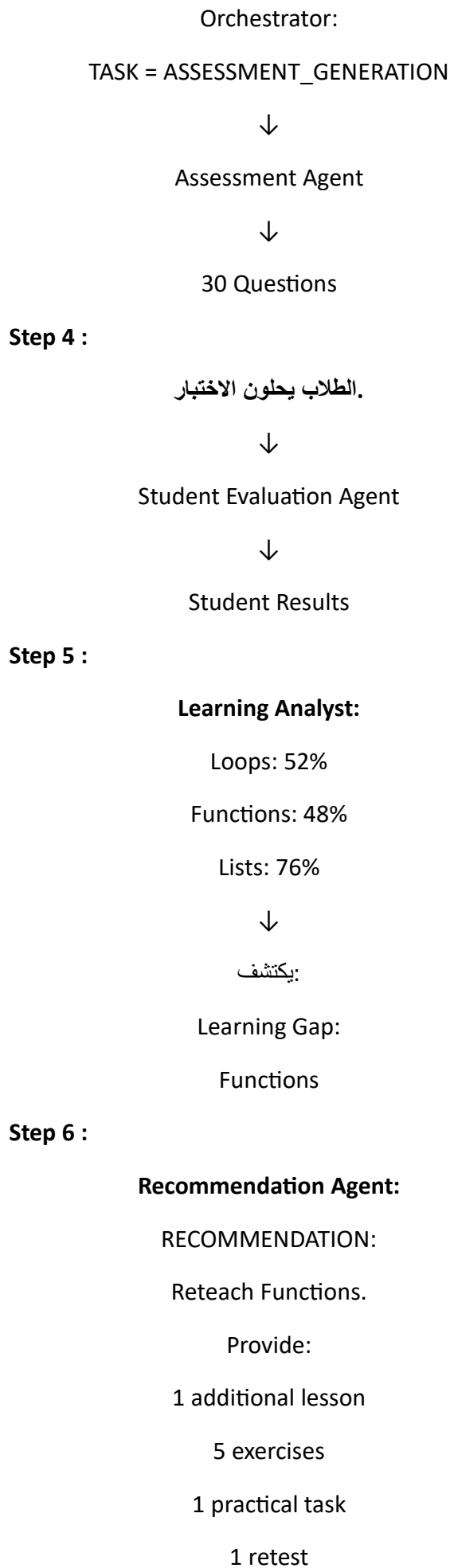
|

| └─ Learning Objectives

|

└─ Assessments

|



```
students: list[Student] = []
assessments: list[Assessment] = []
results: list[StudentResult] = []
recommendations: list[Recommendation] = []
```

وبالتالي يمكن أن ينتقل الـ Agent من مرحلة إلى أخرى.

14. كامل Workflow مثال

لنأخذ:

Python Programming : 30 Hours

يبدأ المستخدم:

Create a beginner Python course
for 20 students
with 30 training hours.

Step 1 :

Orchestrator:

TASK = COURSE_DESIGN

↓

Course Designer

↓

CoursePlan

Step 2 :

Orchestrator:

TASK = CONTENT_GENERATION

↓

Content Agent

↓

Lessons

Exercises

Examples

Step 3 :

Reason:

Below threshold 60%

Recommendation:

Reteach Functions

Confidence:

0.91

وبذلك يستطيع المدرب معرفة: لماذا اتخذ النظام هذا القرار؟

17. Evaluation للنظام نفسه

هذه نقطة سأعتبرها أساسية في المشروع.

لا يكفي أن نقول: النظام يعمل. بل يجب أن نقيسه. مثلاً:

Course Design Evaluation

Objective Coverage

Module Consistency

Time Allocation

Prerequisite Validity

Assessment Evaluation

Question Relevance

Difficulty

Objective Alignment

Answer Correctness

Student Evaluation

Score Accuracy

Classification Accuracy

Recommendation Quality

Agent Evaluation

Tool-call correctness

Schema validity

Policy compliance

Decision accuracy

15. Human-in-the-Loop

هذه نقطة مهمة جداً في مشروع Agentic AI.

لا يجب أن يكون النظام مستقلاً بالكامل.

مثلاً:

Agent Recommendation

|



| Trainer Approval? |

|

|

|

YES

NO

|

|



Execute

Revise

مثلاً إذا أراد النظام:

تعديل خطة الدورة.

يطلب موافقة المدرب.

وهذا يجعل النظام أكثر أماناً وقابلية للتدقيق.

16. Trace / Audit

بما أننا درست موضوع **Tracing**، يمكننا تسجيل كل قرار. مثلاً

TRACE

Agent:

LearningAnalyst

Input:

Student results

Decision:

Functions mastery = 48%

| └─ analyst.txt

|

| └─ database/

| | └─ models.py

| | └─ database.py

|

| └─ evaluation/

| | └─ evaluator.py

| | └─ test_cases.py

|

| └─ tracing/

| | └─ trace.py

|

| └─ config.py

|

| └─ main.py

|

| └─ README.md

19- التقنية التي أقترح استخدامها :

لنسخة أولى قابلة للتنفيذ:

Component	Technology
Language	Python
Agent Framework	OpenAI Agents SDKأوCrewAI
Schemas	Pydantic
Database	SQLite
LLM	Qwen / OpenAI
API	FastAPI
UI	Streamlit

18. Architecture المقترحة للمشروع

البنية البرمجي للمشروع :

training_agent_system/

|

| └─ agents/

| | └─ orchestrator.py

| | └─ course_designer.py

| | └─ content_generator.py

| | └─ assessment_agent.py

| | └─ evaluation_agent.py

| | └─ learning_analyst.py

| | └─ recommendation_agent.py

|

| └─ tools/

| | └─ course_tools.py

| | └─ student_tools.py

| | └─ assessment_tools.py

| | └─ grading_tools.py

| | └─ report_tools.py

|

| └─ schemas/

| | └─ course.py

| | └─ student.py

| | └─ assessment.py

| | └─ evaluation.py

|

| └─ prompts/

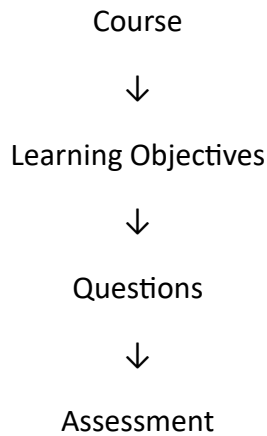
| | └─ orchestrator.txt

| | └─ course_designer.txt

| | └─ assessment.txt

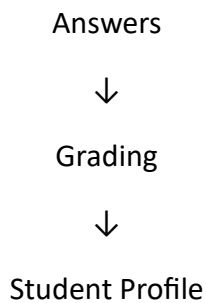
المرحلة 3 :

Assessment :



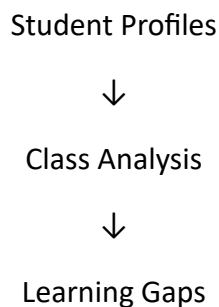
المرحلة 4 :

Student Evaluation :



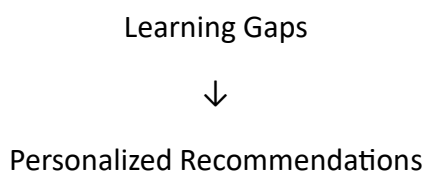
المرحلة 5 :

Learning Analytics:



المرحلة 6 :

Recommendations :



المرحلة 7 : Multi-Agent7

نربط جميع الـ Agents مع Orchestrator.

Component	Technology
Testing	pytest
Tracing	Custom Trace + Agent framework tracing
Documents	PDF/Text parser

لن أن نبدأ مباشرة بـ CrewAI.

الأفضل تعليميًا أن نبني أول نسخة باستخدام

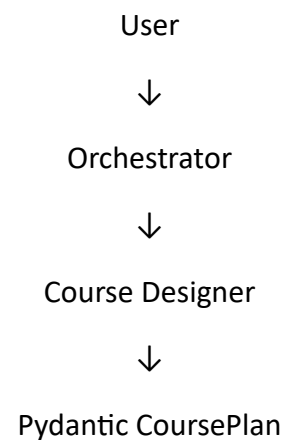
Python + Pydantic + ModelClient

بسيط، ثم نحولها إلى CrewAI. بهذه الطريقة ستفهم ماذا يفعل الـ framework فعليًا بدل أن نعتمد في المشروع على workflow. لأنه يخفي الـ workflow.

مراحل بناء المشروع 20.

لن أقوم ببناء النظام كاملاً دفعة واحدة. بل سأقسمه إلى مراحل :
المرحلة 1 :

Core Agent:

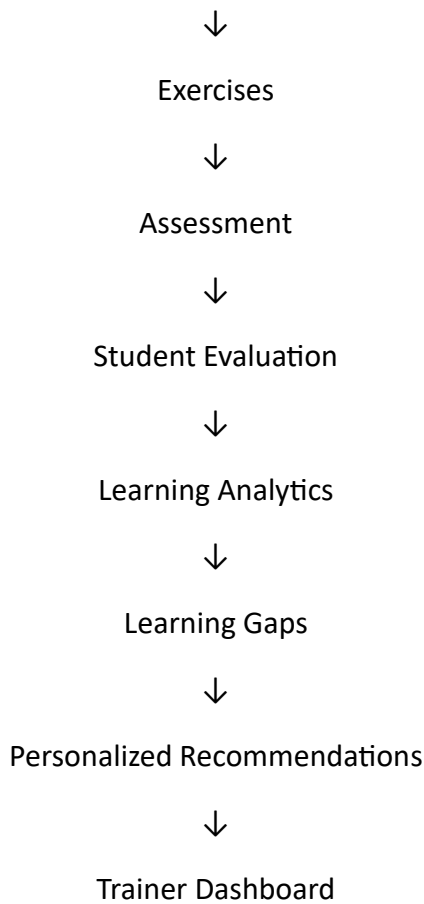


المرحلة 2 :

Tools :

نضيف:

Course DB
Student DB
Grading Tool



UI:

نبنّي Dashboard للمدرّب.

النتيجة النهائية :

سيكون لدينا نظام نستطيع أن نقول له:

"أنشئ دورة Python للمبتدئين، 30 ساعة، لـ 20 طالبًا".

فيقوم النظام تلقائيًا بـ:

Course Design



Learning Objectives



Course Modules



Learning Content

وهذا يعطينا مشروعًا قويًا من ناحية **Agentic AI** لأنه يحتوي على :

planning + multi-agent collaboration + tools + structured outputs + state + memory + human approval + tracing + evaluation.

الخطوة التالية التي ساقوم بها :

الجزء الأهم:

وهو تصميم الـ **System Architecture** بشكل رسمي، ثم نحدد لكل **Agent**:

Goal → Input → Output → Tools → Pydantic Schema → System Prompt → Gates → Failure Conditions → Handoff → Evaluation Criteria

- ساستخدم في المشروع لغة البرمجة **Python** .
- لقد قمت بتقسيم المشروع الى مراحل حتى نقوم ببنائه خطوة بخطوة بدلاً من بناء كود دفعة واحدة ويكون غير مفهوم .